

DBMS – WEEK 6

PET SHOP MANAGEMENT SYSTEM

Product Review and Rating Management System

1. Scenario

The Pet Shop Management System collects customer feedback about pets, pet food, accessories, medicines, grooming products, and other products purchased from the pet shop. After completing an order, customers can provide a rating and review for a purchased product. The system stores customer opinions, analyzes product performance, and identifies highly rated products and products that may require improvement.

For Week 6, a Product Review and Rating Module is created and connected with the existing Customer and Product modules of the Pet Shop Management System.

2. Tasks to be Done

- Create Review and Rating Tables – Store customer feedback and ratings.
- Retrieve Product Review Details – Display and analyze reviews using SQL queries.
- Calculate Average Product Ratings – Use aggregate functions to analyze ratings.
- Identify Highly Rated Products – Generate product rating and customer feedback reports.

3. Create Review and Rating Tables

3.1 Store Customer Feedback and Ratings

The Review table stores the customer, product, rating, review comment, and review date. The Rating column is restricted to values from 1 to 5 using a CHECK constraint.

```
SQL> CREATE TABLE Review (  
  2     Review_ID INT PRIMARY KEY,  
  3     Customer_ID VARCHAR(10),  
  4     Product_ID VARCHAR(10),  
  5     Rating INT CHECK (Rating BETWEEN 1 AND 5),  
  6     Review_Comment VARCHAR(500),  
  7     Review_Date DATE,  
  8     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
  9     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
10 );
```

Table created.

```
SQL> desc review;
```

Name	Null?	Type
REVIEW_ID	NOT NULL	NUMBER(38)
CUSTOMER_ID		VARCHAR2(10)
PRODUCT_ID		VARCHAR2(10)
RATING		NUMBER(38)
REVIEW_COMMENT		VARCHAR2(500)
REVIEW_DATE		DATE

3.2 Add Reviews for Purchased Products

Example review records for products in the Pet Shop Management System:

```
SQL> INSERT INTO Review
2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
3 VALUES
4 (1, 'S101', 'P101', 5, 'Excellent quality dog food.', DATE '2026-09-20');

1 row created.

SQL>
SQL> INSERT INTO Review
2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
3 VALUES
4 (2, 'S102', 'P102', 4, 'My cat liked this product.', DATE '2026-09-21');

1 row created.

SQL>
SQL> INSERT INTO Review
2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
3 VALUES
4 (3, 'S103', 'P103', 5, 'Good quality and useful accessory.', DATE '2026-09-22');

1 row created.

SQL>
SQL> INSERT INTO Review
2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
3 VALUES
4 (4, 'S104', 'P104', 3, 'Average product; packaging can improve.', DATE '2026-09-23');

1 row created.

SQL>
SQL> INSERT INTO Review
2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
3 VALUES
4 (5, 'S105', 'P105', 2, 'Product quality needs improvement.', DATE '2026-09-24');

1 row created.

SQL>
SQL> INSERT INTO Review
2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
3 VALUES
4 (6, 'S106', 'P106', 5, 'Very useful and good quality.', DATE '2026-09-25');

1 row created.
```

```
SQL>
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (7, 'S107', 'P107', 4, 'Good product for the price.', DATE '2026-09-26');

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (8, 'S108', 'P108', 3, 'Product is okay but can be better.', DATE '2026-09-27');

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (9, 'S109', 'P109', 5, 'Highly satisfied with the product.', DATE '2026-09-28');

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (10, 'S110', 'P110', 4, 'Good quality and fast results.', DATE '2026-09-29');

1 row created.
```

```
SQL> SELECT * FROM Review;
```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING	REVIEW_COMMENT	REVIEW_DATE
1	S101	P101	5	Excellent quality dog food.	20-SEP-26
2	S102	P102	4	My cat liked this product.	21-SEP-26
3	S103	P103	5	Good quality and useful accessory.	22-SEP-26
4	S104	P104	3	Average product; packaging can improve.	23-SEP-26
5	S105	P105	2	Product quality needs improvement.	24-SEP-26
6	S106	P106	5	Very useful and good quality.	25-SEP-26
7	S107	P107	4	Good product for the price.	26-SEP-26
8	S108	P108	3	Product is okay but can be better.	27-SEP-26
9	S109	P109	5	Highly satisfied with the product.	28-SEP-26
10	S110	P110	4	Good quality and fast results.	29-SEP-26

10 rows selected.

3.3 Store Customer Ratings from 1 to 5 Stars

The CHECK constraint in the Review table ensures that only ratings between 1 and 5 can be stored.

```
SQL> INSERT INTO Review
2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
3 VALUES
4 (11, 'S101', 'P101', 5, 'Excellent product.', DATE '2026-09-30');

1 row created.
```

3.4 Update Review Comments if Required

```
SQL> UPDATE Review
  2 SET Review_Comment = 'Good product and my pet likes it.'
  3 WHERE Review_ID = 2;

1 row updated.
```

3.5 Remove Inappropriate or Invalid Reviews

```
SQL> DELETE FROM Review
  2 WHERE Review_ID = 5;

1 row deleted.
```

4. Retrieve Product Review Details

Write SQL queries to retrieve and display customer review information.

4.1 Display All Reviews for a Product

```
SQL> SELECT *
  2 FROM Review
  3 WHERE Product_ID = 'P101';
```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING	REVIEW_COMMENT
1	S101	P101	5	Excellent quality dog food.
11	S101	P101	5	Excellent product.

4.2 Display Customer Name with Their Reviews

```

SQL> SELECT c.First_Name,
2         c.Last_Name,
3         r.Product_ID,
4         r.Rating,
5         r.Review_Comment,
6         r.Review_Date
7 FROM Customer c
8 JOIN Review r
9 ON c.Customer_ID = r.Customer_ID;

```

FIRST_NAME	LAST_NAME	PRODUCT_ID
AKSHAYA	NIVASINI	P101
5	Excellent quality dog food.	20-SEP-26
PRITHISH	RAJ	P102
4	Good product and my pet likes it.	21-SEP-26
KSHAUNISH	RAJESH	P103
5	Good quality and useful accessory.	22-SEP-26

FIRST_NAME	LAST_NAME	PRODUCT_ID
KANISKA	RAM	P104
3	Average product; packaging can improve.	23-SEP-26
VIJAY	DEEPIKA	P106
5	Very useful and good quality.	25-SEP-26
AURA	NIVAS	P107
4	Good product for the price.	26-SEP-26

FIRST_NAME	LAST_NAME	PRODUCT_ID
SHREYA	RAMAN	P108
3	Product is okay but can be better.	27-SEP-26
GEETHA	GOVIND	P109
5	Highly satisfied with the product.	28-SEP-26
RAMYA	KRISHNAN	P110
4	Good quality and fast results.	29-SEP-26

FIRST_NAME	LAST_NAME	PRODUCT_ID
AKSHAYA	NIVASINI	P101
5	Excellent product.	30-SEP-26

10 rows selected.

4.3 Find Products Having Maximum Reviews

```
SQL> SELECT Product_ID,  
2      COUNT(*) AS Number_of_Reviews  
3 FROM Review  
4 GROUP BY Product_ID  
5 ORDER BY Number_of_Reviews DESC;
```

PRODUCT_ID	NUMBER_OF_REVIEWS
P101	2
P107	1
P102	1
P110	1
P108	1
P106	1
P104	1
P109	1
P103	1

9 rows selected.

4.4 Display Recent Customer Feedback

```

SQL> SELECT Review_ID,
2         Customer_ID,
3         Product_ID,
4         Rating,
5         Review_Comment,
6         Review_Date
7 FROM Review
8 ORDER BY Review_Date DESC;

```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING	REVIEW_COMMENT
11	S101	P101	5	Excellent product.
10	S110	P110	4	Good quality and fast results.
9	S109	P109	5	Highly satisfied with the product.
8	S108	P108	3	Product is okay but can be better.
7	S107	P107	4	Good product for the price.
6	S106	P106	5	Very useful and good quality.
4	S104	P104	3	Average product; packaging can improve.
3	S103	P103	5	Good quality and useful accessory.
2	S102	P102	4	Good product and my pet likes it.
1	S101	P101	5	Excellent quality dog food.

10 rows selected.

4.5 Retrieve Reviews with Ratings Above 4

```
SQL> SELECT *
  2   FROM Review
  3   WHERE Rating > 4;
```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	RATING	REVIEW_COMMENT
20-SEP-26	1 S101	P101	5	Excellent quality dog food.
22-SEP-26	3 S103	P103	5	Good quality and useful accessory.
25-SEP-26	6 S106	P106	5	Very useful and good quality.
28-SEP-26	9 S109	P109	5	Highly satisfied with the product.
30-SEP-26	11 S101	P101	5	Excellent product.

5. Calculate Average Product Ratings

Aggregate functions are used to analyze product ratings and understand how customers are responding to products sold by the pet shop.

5.1 Calculate Average Rating for Each Product

```
SQL> SELECT Product_ID,
  2          AVG(Rating) AS Average_Rating
  3   FROM Review
  4   GROUP BY Product_ID;
```

PRODUCT_ID	AVERAGE_RATING
P109	5
P107	4
P102	4
P110	4
P103	5
P106	5
P104	3
P101	5
P108	3

9 rows selected.

5.2 Count the Number of Reviews per Product

```
SQL> SELECT Product_ID,  
2          COUNT(*) AS Number_of_Reviews  
3 FROM Review  
4 GROUP BY Product_ID;
```

PRODUCT_ID	NUMBER_OF_REVIEWS
P109	1
P107	1
P102	1
P110	1
P103	1
P106	1
P104	1
P101	2
P108	1

9 rows selected.

5.3 Find Highest-Rated Products

```
SQL> SELECT Product_ID,  
2          AVG(Rating) AS Average_Rating  
3 FROM Review  
4 GROUP BY Product_ID  
5 ORDER BY Average_Rating DESC;
```

PRODUCT_ID	AVERAGE_RATING
P109	5
P103	5
P101	5
P106	5
P110	4
P102	4
P107	4
P108	3
P104	3

9 rows selected.

5.4 Identify Products with Average Rating Above 4

```
SQL> SELECT Product_ID,  
2         AVG(Rating) AS Average_Rating  
3 FROM Review  
4 GROUP BY Product_ID  
5 HAVING AVG(Rating) > 4;
```

PRODUCT_ID	AVERAGE_RATING
P109	5
P103	5
P106	5
P101	5

6. Identify Highly Rated Products

The following reports summarize product performance and customer feedback for the Pet Shop Management System.

Report 1: Product Rating Analysis

Display:

- Product name
- Number of reviews
- Average rating

```
SQL> SELECT p.Product_Name,
2         COUNT(r.Review_ID) AS Number_of_Reviews,
3         ROUND(AVG(r.Rating), 2) AS Average_Rating
4 FROM Product p
5 JOIN Review r
6 ON p.Product_ID = r.Product_ID
7 GROUP BY p.Product_ID, p.Product_Name
8 ORDER BY Average_Rating DESC;
```

PRODUCT_NAME

NUMBER_OF_REVIEWS AVERAGE_RATING

pet vitamin supplement

1 5

pedigree dog food

2 5

cat ball toy

1 5

PRODUCT_NAME

NUMBER_OF_REVIEWS AVERAGE_RATING

bird seed mix

1 5

pet sleeping bed

1 4

cat dry food

1 4

PRODUCT_NAME

NUMBER_OF_REVIEWS AVERAGE_RATING

pet shampoo

1 4

Dog collar

1 3

fish food flakes

1 3

9 rows selected.

Report 2: Customer Feedback Analysis

Find:

- Most reviewed products
- Highly rated products
- Products requiring improvement

Most reviewed products:

```

SQL> SELECT p.Product_Name,
2      COUNT(r.Review_ID) AS Number_of_Reviews
3 FROM Product p
4 JOIN Review r
5 ON p.Product_ID = r.Product_ID
6 GROUP BY p.Product_ID, p.Product_Name
7 ORDER BY Number_of_Reviews DESC;

```

PRODUCT_NAME

NUMBER_OF_REVIEWS

pedigree dog food
2

pet sleeping bed
1

cat dry food
1

PRODUCT_NAME

NUMBER_OF_REVIEWS

cat ball toy
1

Dog collar
1

bird seed mix
1

PRODUCT_NAME

NUMBER_OF_REVIEWS

pet vitamin supplement
1

pet shampoo
1

fish food flakes
1

9 rows selected.

Highly rated products:

```

SQL> SELECT p.Product_Name,
2         ROUND(AVG(r.Rating), 2) AS Average_Rating
3 FROM Product p
4 JOIN Review r
5 ON p.Product_ID = r.Product_ID
6 GROUP BY p.Product_ID, p.Product_Name
7 HAVING AVG(r.Rating) > 4
8 ORDER BY Average_Rating DESC;

```

PRODUCT_NAME

AVERAGE_RATING

pedigree dog food
5

cat ball toy
5

pet vitamin supplement
5

PRODUCT_NAME

AVERAGE_RATING

bird seed mix
5

Products requiring improvement:

```

SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (12, 'S105', 'P105', 2, 'Product quality needs improvement.', DATE '2026-09-30');

1 row created.

SQL> commit;

Commit complete.

SQL> SELECT p.Product_Name,
  2         ROUND(AVG(r.Rating), 2) AS Average_Rating
  3 FROM Product p
  4 JOIN Review r
  5 ON p.Product_ID = r.Product_ID
  6 GROUP BY p.Product_ID, p.Product_Name
  7 HAVING AVG(r.Rating) < 3
  8 ORDER BY Average_Rating ASC;

PRODUCT_NAME
-----
AVERAGE_RATING
-----
puppy chew toy
                2

```

Report 3: Rating Distribution

Display:

- Number of 5-star ratings
- Number of 4-star ratings
- Number of low-rated products
- Complete rating distribution

Number of 5-star ratings:

```

SQL> SELECT COUNT(*) AS Five_Star_Ratings
  2 FROM Review
  3 WHERE Rating = 5;

FIVE_STAR_RATINGS
-----
                    5

```

Number of 4-star ratings:

```

SQL> SELECT COUNT(*) AS Four_Star_Ratings
  2 FROM Review
  3 WHERE Rating = 4;

FOUR_STAR_RATINGS
-----
                    3

```

Number of low-rated products:

```
SQL> SELECT COUNT(DISTINCT Product_ID) AS Low_Rated_Products
  2   FROM Review
  3   WHERE Rating <= 2;

LOW_RATED_PRODUCTS
-----
                  1
```

Complete rating distribution:

```
SQL> SELECT Rating,
  2         COUNT(*) AS Number_of_Ratings
  3   FROM Review
  4   GROUP BY Rating
  5   ORDER BY Rating DESC;

RATING  NUMBER_OF_RATINGS
-----
        5              5
        4              3
        3              2
        2              1
```

7. Expected Outcome

Module / Report	Expected Result
Review and Rating Table	Stores customer reviews and ratings from 1 to 5.
Product Review Details	Retrieves reviews, customer names, recent feedback and highly rated reviews.
Average Product Ratings	Calculates average ratings and review counts for each product.
Highly Rated Products	Identifies products with average rating above 4.
Customer Feedback Analysis	Finds most reviewed, highly rated and improvement-required products.
Rating Distribution	Shows the number of 5-star, 4-star, low-rated and complete rating results.

8. Conclusion

The Product Review and Rating Management System adds a customer-feedback component to the Pet Shop Management System. It allows the shop to store valid ratings and review comments, retrieve customer feedback, calculate product rating statistics, and identify highly rated and low-rated products. These reports help evaluate product performance and understand customer opinions.